Project Report

Master of Computer Application
Semester – II
Java Object-Oriented Design and Programming

Project title: **Hospital Management System**

Submitted By:

1. **Mohd Nadeem – Reg. No.2511022250019**
2. **Priyanshu Kumar – Reg. No. 2511022250002**
3. **Vishal Kumar – Reg. No.2511022250027**
4. **Mohammed Shabaz khan – Reg. No.2511022250091**

Submitted To:

Faculty Name: Mary Agnes J

Mentor/Faculty Signature:

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road,
Anekal Bengaluru - 562 106

March 2025

Table of Contents

This report covers all aspects of the PulseCare 2026 Hospital Management System project, developed as part of the Java Object-Oriented Designing and Programming course.

Section	Title
1	About the Course
2	System Requirements
3	Introduction About the Project
4	Technologies Used & Possible Enhancements
5	Objectives
6	Implementation
7	Output Screenshots
8	Future Enhancements
9	Conclusion
10	References

1. About the Course

Java Object-Oriented Designing and Programming

Java Object-Oriented Designing and Programming is a foundational course in the field of computer science and software engineering. The course introduces students to the principles and practices of object-oriented programming (OOP) using the Java programming language, one of the most widely adopted, platform-independent, and enterprise-grade programming languages in the industry.

Core Principles Covered

- Encapsulation: Bundling data (fields) and methods that operate on the data into a single unit called a class, restricting direct access to internal components using access modifiers such as private, protected, and public.
- Inheritance: Enabling a new class (subclass) to acquire the properties and behaviors of an existing class (superclass), promoting code reuse and hierarchical class structures.
- Polymorphism: Allowing objects to be treated as instances of their parent class, and enabling one interface to be used for a general class of actions, through method overloading and method overriding.
- Abstraction: Hiding complex implementation details and exposing only the necessary functionality to the user, achieved through abstract classes and interfaces.

Programming Concepts Taught

- Java Syntax and Semantics: Variables, data types, operators, control flow structures (if-else, loops, switch).
- Classes and Objects: Constructors, instance variables, static members, and object lifecycle.
- Exception Handling: Try-catch-finally blocks, custom exceptions, checked and unchecked exceptions.
- Collections Framework: ArrayList, LinkedList, HashMap, HashSet, and iteration patterns.
- Swing GUI Programming: Building desktop applications with JFrame, JPanel, JButton, JTable, JTextField, JComboBox, and event listeners.
- Event-Driven Programming: ActionListener, MouseListener, and Lambda expressions for concise event handling.
- Data Modeling: Designing model classes with proper field types, constructors, and getter methods.

The course emphasizes not only writing functional code but also designing software with proper architecture, clean code practices, and maintainability in mind. Students are expected to produce real-world-style projects that demonstrate mastery of OOP concepts in Java.

2. System Requirements

Hardware Requirements

The following minimum hardware specifications are recommended to compile, run, and demonstrate the PulseCare 2026 Hospital Management System application effectively:

Component	Minimum Specification
Processor	Intel Core i3 (8th Gen) or AMD Ryzen 3 / equivalent at 2.0 GHz or higher
RAM	4 GB (8 GB Recommended for smooth UI rendering)
Storage	256 MB free disk space for JDK, project files, and compiled class files
Display	1280 x 720 pixels minimum; 1920 x 1080 recommended for full UI visibility
Input Devices	Standard keyboard and mouse
Graphics	Integrated graphics card sufficient (no dedicated GPU required)
Network	Not required (standalone desktop application)

Software Requirements

The following software must be installed and properly configured on the development or demonstration machine:

Software	Version / Details
Operating System	Windows 10 / 11, macOS 12+, or Ubuntu 20.04 LTS and above
Java Development Kit (JDK)	JDK 17 or JDK 21 (LTS versions recommended)
Java Runtime Environment (JRE)	Bundled with JDK 17+ (no separate installation needed)
IDE (Recommended)	IntelliJ IDEA Community Edition 2023+ or Eclipse IDE 2023+
Build Tool	Command-line javac and java (no Maven/Gradle required for this project)
Terminal / CMD	Windows CMD, PowerShell, or Unix terminal for running run.bat
Version Control	Git (optional, for source code management)

Environment Setup Instructions

- Step 1: Download and install JDK 17 or later from https://adoptium.net or Oracle's official website.
- Step 2: Set the JAVA_HOME environment variable to the JDK installation directory.
- Step 3: Add the JDK's bin folder to the system PATH variable.
- Step 4: Verify the installation by running java -version and javac -version in terminal.
- Step 5: Navigate to the project root directory and execute run.bat (Windows) or manually compile with javac -d out src/hospital/*.java and run with java -cp out hospital.Main.

3. Introduction About the Project

3.1 Purpose

PulseCare 2026 is a fully functional desktop Hospital Management System developed using Java Swing, designed to modernize and streamline the administrative operations of a hospital or medical facility. The primary purpose of this project is to eliminate the inefficiencies associated with paper-based hospital record keeping by providing a clean, intuitive, and real-time digital interface for front-desk personnel, administrative staff, and management.

The application enables hospital staff to register and manage patients, maintain a comprehensive doctor directory, and schedule appointments seamlessly. All data operations are performed in-memory during the session, making the system lightweight and instantly deployable without any database configuration. The project was developed as the final submission for the Java Object-Oriented Designing and Programming course and serves as a demonstration of advanced Swing UI techniques combined with solid OOP architecture.

3.2 Scope

The scope of PulseCare 2026 encompasses the following operational areas within a hospital administration context:

- Patient Management: Complete patient registration, search, filter, and removal capabilities with fields for name, age, gender, phone number, and diagnosed condition.
- Doctor Directory Management: Maintaining a comprehensive roster of doctors with their specialization departments and weekly availability schedules.
- Appointment Scheduling: Linking patients to doctors with date and status tracking (Scheduled, Pending, Completed).
- Dashboard Analytics: Real-time numerical statistics for total patients, doctors, appointments, and estimated occupancy rate.
- Department Highlights: Visual overview of active hospital departments including Cardiology, Neurology, Orthopedics, Pediatrics, Dermatology, and Emergency Medicine.

The system is intentionally scoped as a standalone desktop application without network connectivity, external database integration, or multi-user authentication, making it ideal for academic demonstration and rapid prototyping purposes.

3.3 Key Features

- Gradient Hero Banner: A visually striking purple-to-pink gradient header panel with the brand name, tagline, and quick-access action buttons (New Patient, Book Visit, Refresh).

- Tabbed Navigation: Four clearly separated tabs — Dashboard, Patients, Doctors, and Appointments — each with dedicated form panels and sortable, searchable data tables.
- Shift Snapshot Sidebar: A persistent left sidebar displaying real-time metrics (patient count, doctor count, appointment count, occupancy percentage) with color-coded cards.
- Live Search and Filtering: Each data table supports regex-based live search by keyword and dropdown filtering by gender, department, or status simultaneously.
- Color-coded Appointment Status: The appointments table renders status badges in distinct colors — green for Scheduled, amber for Pending, and blue for Completed.
- Department Drill-Down: Clickable department cards on the dashboard automatically switch to the Doctors tab and filter by the selected department.
- Form Validation: All input forms validate for empty fields and numeric constraints before adding records, displaying friendly warning dialogs.
- Seeded Demo Data: The application launches with 20 pre-loaded doctors and 2 patients with appointments, ensuring a populated, demo-ready state on startup.
- Striped Table Rows: Alternating row backgrounds for improved readability across all data tables.

4. Technologies Used & Possible Enhancements

4.1 Technologies Used

Technology	Description	Usage in Project
Java SE 17+	Core programming language	All application logic, OOP modeling
Java Swing	GUI toolkit included in JDK	All UI components and windows
javax.swing.JFrame	Top-level window container	Main application window
javax.swing.JTabbedPane	Tab-based navigation	Dashboard, Patients, Doctors, Appointments
javax.swing.JTable	Data grid component	Displaying records in tabular format
javax.swing.DefaultTableModel	Table data management	Adding, removing, refreshing rows
javax.swing.TableRowSorter	Row sorting and filtering	Live search and filter functionality
java.awt.GradientPaint	Gradient rendering	Hero banner background gradient
java.awt.Graphics2D	2D drawing API	Custom painted gradient panel
java.util.ArrayList	Dynamic list collection	In-memory storage of patients, doctors, appointments
javax.swing.JOptionPane	Dialog boxes	Success and validation messages

4.2 Possible Enhancements

While the current system fulfills its academic objectives comprehensively, the following enhancements could be incorporated in future production-grade versions:

- Database Integration: Replacing in-memory ArrayList storage with a persistent relational database such as MySQL or PostgreSQL using JDBC, or an embedded database like SQLite for portability.
- User Authentication: Implementing a login screen with role-based access control (Admin, Doctor, Receptionist) to restrict sensitive operations.

- PDF Report Generation: Adding functionality to export patient records, doctor schedules, and appointment summaries as PDF documents using libraries such as iTextPDF or Apache PDFBox.
- Electronic Health Records (EHR): Extending the Patient model to include medical history, prescription records, lab results, and allergies.
- Billing and Invoice Module: Implementing a billing tab for generating invoices for consultations, procedures, and medications.
- Notifications and Reminders: Sending automated appointment reminders via email using JavaMail API or SMS integrations.

5. Objectives

The primary objectives of the PulseCare 2026 project are outlined as follows:

- Demonstrate Mastery of OOP Principles: Apply the four pillars of object-oriented programming — Encapsulation, Inheritance, Polymorphism, and Abstraction — in a real-world application context through a well-structured multi-class Java project.
- Design and Implement a Functional GUI Application: Build a professional-grade desktop application using Java Swing that goes beyond basic UI components, incorporating custom painted panels, gradient effects, tabbed navigation, styled tables, and responsive layouts.
- Implement CRUD Operations: Develop complete Create, Read, Update (via refresh), and Delete functionality for three core entities — Patients, Doctors, and Appointments — using in-memory data structures.
- Implement Real-Time Search and Filtering: Enable hospital staff to quickly locate records using live keyword search combined with categorical dropdown filters, powered by Swing's TableRowSorter and RowFilter APIs.
- Model Real-World Healthcare Entities: Design well-encapsulated model classes (Patient, Doctor, Appointment) that accurately represent real healthcare data with appropriate attributes and immutable getter methods.
- Build an Intuitive User Interface: Create a user experience that is clean, modern, and easy to use for non-technical hospital staff, using clear labels, color-coded status indicators, organized form layouts, and informative dashboard widgets.
- Demonstrate Data Relationships: Show the relationship between patients, doctors, and appointments by dynamically populating the doctor selection in appointment forms and cascading doctor deletions to related appointments.
- Produce a Complete Academic Submission: Deliver a fully documented, well-structured, and compilable Java project that meets all academic requirements for the Java OOP course.

6. Implementation

6.1 Project Structure

The project follows a clean package-based structure with all source files organized under the hospital package:

File	Purpose
src/hospital/Main.java	Application entry point; initializes UI with system look-and-feel
src/hospital/Patient.java	Model class representing a hospital patient
src/hospital/Doctor.java	Model class representing a hospital doctor
src/hospital/Appointment.java	Model class representing a scheduled appointment
src/hospital/HospitalManagementUI.java	Main UI class extending JFrame; all visual logic

6.2 Module Descriptions

Module 1: Patient Model (Patient.java)

The Patient class is a fully encapsulated data model representing a hospital patient. It uses the final keyword for all fields to enforce immutability once a patient object is created. All fields are private and accessible only through public getter methods, strictly adhering to the encapsulation principle of OOP.

Fields: id (String), name (String), age (int), gender (String), phone (String), disease (String)

Methods: getId(), getName(), getAge(), getGender(), getPhone(), getDisease()

Module 2: Doctor Model (Doctor.java)

The Doctor class encapsulates information about a medical professional in the hospital. Similar to Patient, it uses final fields and provides public getter methods. Each doctor carries a unique ID, name, department, and availability schedule string.

Fields: id (String), name (String), department (String), availability (String)

Methods: getId(), getName(), getDepartment(), getAvailability()

Module 3: Appointment Model (Appointment.java)

The Appointment class links patients to doctors for a scheduled visit. It stores the appointment ID, patient name, doctor name, visit date, and current status. The status field can hold Scheduled, Pending, or Completed values as set during booking.

Fields: appointmentId (String), patientName (String), doctorName (String), date (String), status (String)

Methods: getAppointmentId(), getPatientName(), getDoctorName(), getDate(), getStatus()

Module 4: Main Entry Point (Main.java)

The Main class contains the application entry point. It attempts to apply the system's native look-and-feel using UIManager.setLookAndFeel() for a platform-consistent appearance. It then uses SwingUtilities.invokeLater() to ensure the GUI is created on the Event Dispatch Thread (EDT) — a critical best practice in Swing development.

Module 5: Hospital Management UI (HospitalManagementUI.java)

This is the central and most comprehensive module of the project. It extends JFrame and manages all visual components, user interactions, and business logic. The class is structured into logically separated private methods for clarity and maintainability:

- hero(): Builds the gradient banner with brand name, tagline, and action buttons using a custom GradientPanel inner class.
- sidebar(): Constructs the Shift Snapshot panel with color-coded metric cards and a live desk note.
- mainTabs(): Creates the JTabbedPane and adds all four tab pages.
- dashboard(): Builds the Dashboard tab with stat cards and department highlight buttons.
- patientsPage(), doctorsPage(), appointmentsPage(): Build their respective tab pages with form panels and table panels.
- addPatient(), addDoctor(), addAppointment(): Handle form submissions with validation, object creation, and table refresh.
- removePatient(), removeDoctor(), removeAppointment(): Handle record deletion with cascading logic for doctors.
- applyPatientFilter(), applyDoctorFilter(), applyAppointmentFilter(): Apply combined keyword and categorical filters using RowFilter.
- styleTable(): Applies consistent visual styling to all JTable instances with custom cell renderers and status-based color coding.
- seed(): Pre-populates 20 doctors and 2 patients with 2 appointments for immediate demo readiness.

6.3 Relevant Code Snippets

Patient.java - Encapsulated Model Class

```java
package hospital;

public class Patient {
    private final String id;
    private final String name;
    private final int age;
    private final String gender;
    private final String phone;
    private final String disease;

    public Patient(String id, String name, int age,
                   String gender, String phone, String disease) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.gender = gender;
        this.phone = phone;
        this.disease = disease;
    }

    public String getId()     { return id; }
    public String getName()   { return name; }
    public int    getAge()    { return age; }
    public String getGender() { return gender; }
    public String getPhone()  { return phone; }
    public String getDisease(){ return disease; }
}
```

HospitalManagementUI.java - addPatient() Method

```java
private void addPatient() {
    if (pName.getText().trim().isEmpty() ||
        pAge.getText().trim().isEmpty()  ||
        pPhone.getText().trim().isEmpty() ||
        pDisease.getText().trim().isEmpty()) {
        warn("Please fill all patient fields.");
        return;
    }
    int age;
    try {
        age = Integer.parseInt(pAge.getText().trim());
    } catch (NumberFormatException ex) {
        warn("Age must be a valid number.");
        return;
    }
    Patient patient = new Patient(
        "P" + patientCounter++,
        pName.getText().trim(), age,
        pGender.getSelectedItem().toString(),
        pPhone.getText().trim(),
        pDisease.getText().trim()
    );
    patients.add(patient);
    patientModel.addRow(new Object[]{
        patient.getId(), patient.getName(), patient.getAge(),
        patient.getGender(), patient.getPhone(), patient.getDisease()
    });
```

```
    clearPatient();
    refreshMetrics();
    JOptionPane.showMessageDialog(this, "Patient added successfully.");
}
```

Custom GradientPanel - Inner Class

```
private static class GradientPanel extends JPanel {
    private final Color start, end;
    GradientPanel(BorderLayout layout, Color start, Color end) {
        super(layout);
        this.start = start;
        this.end = end;
        setOpaque(false);
    }
    @Override
    protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                            RenderingHints.VALUE_ANTIALIAS_ON);
        g2.setPaint(new GradientPaint(
            0, 0, start, getWidth(), getHeight(), end));
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 36, 36);
        g2.dispose();
        super.paintComponent(g);
    }
}
```

Live Filter Logic - applyPatientFilter()

```
private void applyPatientFilter() {
    List<RowFilter<Object, Object>> filters = new ArrayList<>();
    String search = patientSearch.getText().trim();
    String gender = patientFilter.getSelectedItem() == null
        ? "All Genders"
        : patientFilter.getSelectedItem().toString();
    if (!search.isEmpty())
        filters.add(RowFilter.regexFilter(
            "(?i)" + Pattern.quote(search)));
    if (!"All Genders".equals(gender))
        filters.add(RowFilter.regexFilter(
            "^" + Pattern.quote(gender) + "$", 3));
    patientSorter.setRowFilter(
        filters.isEmpty() ? null : RowFilter.andFilter(filters));
}
```

6.4 Module Responsibilities

The following table outlines the recommended distribution of responsibilities among team members:

Module	Description	Assigned To

Patient.java	Patient model class design and encapsulation	Team Member 1
Doctor.java	Doctor model class design and encapsulation	Team Member 1
Appointment.java	Appointment model class design	Team Member 2
Main.java	Application entry point and EDT setup	Team Member 2
Hero Banner UI	GradientPanel, title, action buttons	Team Member 3
Sidebar / Shift Snapshot	Metric cards and desk note panel	Team Member 3
Dashboard Tab	Stat cards and department highlights	Team Member 1
Patients Tab (Form + Table)	Patient intake form and records table	Team Member 2
Doctors Tab (Form + Table)	Doctor roster form and directory table	Team Member 3
Appointments Tab	Visit planner form and timeline table	Team Member 1
Search & Filter Logic	applyFilter() methods for all three tabs	Team Member 2
Table Styling & Rendering	styleTable(), custom cell renderer	Team Member 3
seed() Data Population	Pre-loading demo doctors, patients, appointments	Team Member 1
Testing & Integration	End-to-end testing, bug fixes, documentation	All Members

7. Output Screenshots

7.1 Application Dashboard - Main View

The following screenshot shows the PulseCare 2026 application immediately after launch. The dashboard is populated with seeded demo data showing 2 patients, 20 doctors, and 2 appointments. The gradient hero banner, tabbed navigation, sidebar metrics, department highlights, and stat cards are all visible in their default state.

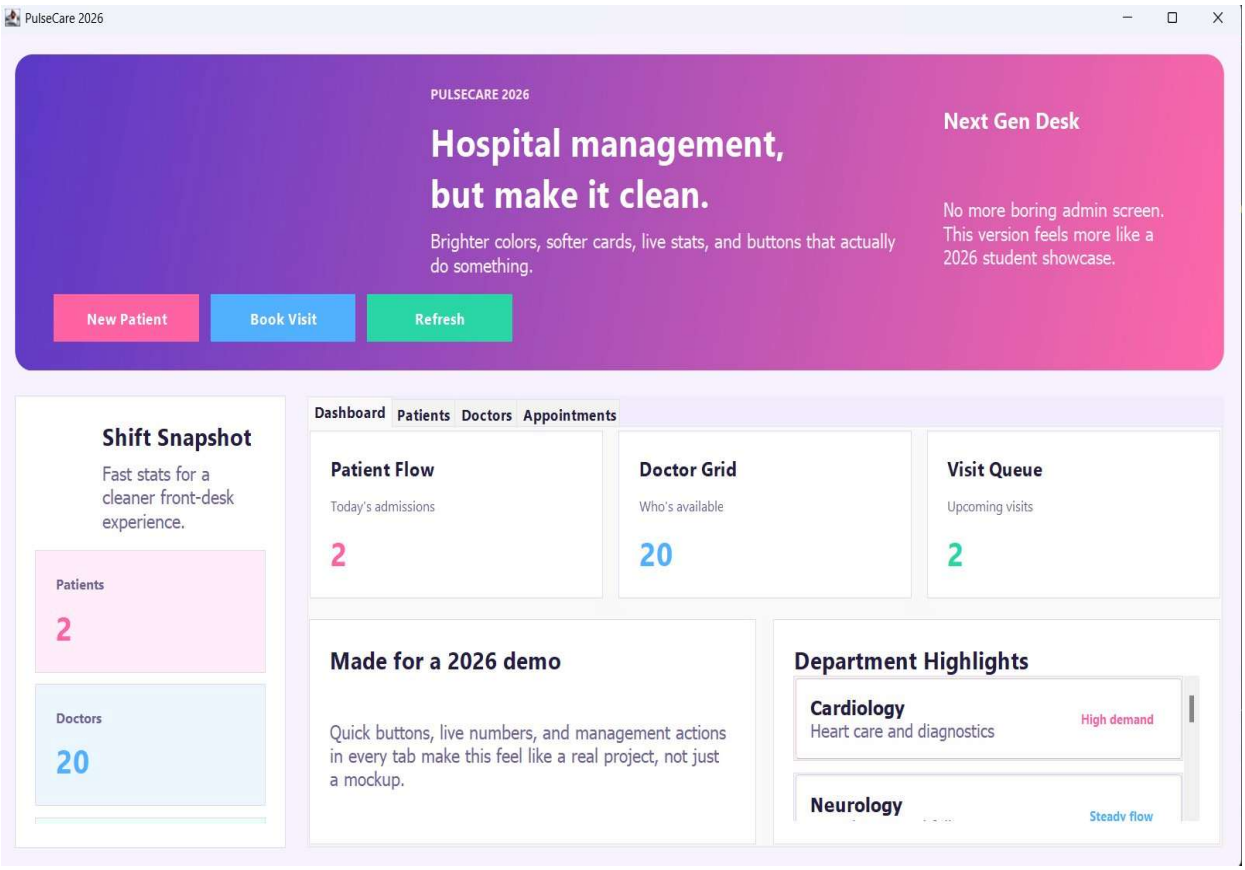

Figure 7.1: PulseCare 2026 - Main Dashboard View showing Hero Banner, Shift Snapshot Sidebar, and Department Highlights

7.2 Dashboard Features Visible

- Hero Banner (Top): Purple-to-pink gradient panel with 'Hospital management, but make it clean.' tagline and three action buttons: New Patient (pink), Book Visit (blue), and Refresh (green).
- Shift Snapshot Sidebar (Left): Color-coded metric tiles showing Patients: 2 (pink), Doctors: 20 (blue), and Appointments with live occupancy percentage.
- Stat Cards (Center Top): Patient Flow (2 today), Doctor Grid (20 available), Visit Queue (2 upcoming).

- Department Highlights (Right): Scrollable cards for Cardiology (High demand), Neurology (Steady flow), Orthopedics (Moderate load), Pediatrics, Dermatology, and Emergency Medicine — each clickable to filter doctors by department.
- Tab Navigation: Dashboard, Patients, Doctors, and Appointments tabs clearly visible at the top of the content area.

7.3 Other Application Views (Described)

Patients Tab

The Patients tab presents a two-panel layout. The left panel (Patient Intake form) contains labeled input fields for Patient Name, Age, Gender (dropdown), Phone, and Condition, along with Add Patient (pink), Clear Form (ghost), and Remove Selected (ghost) buttons. The right panel displays the Patient Records table with columns: ID, Patient, Age, Gender, Phone, and Condition. Above the table, a search bar with a keyword input, gender filter dropdown, Apply, and Reset buttons enable live filtering.

Doctors Tab

The Doctors tab mirrors the Patients layout with a Doctor Roster form (Name, Department, Availability) and the Doctor Directory table. The doctor directory is pre-populated with 20 doctors spanning 20 departments from Cardiology to Physiotherapy. The department filter dropdown dynamically populates based on existing doctor entries.

Appointments Tab

The Appointments tab features the Visit Planner form linking patient names (text input) to doctors (dropdown populated from the doctor list) with a date field and status dropdown (Scheduled, Pending, Completed). The Appointment Timeline table color-codes status cells: green for Scheduled, amber for Pending, and blue for Completed.

8. Future Enhancements

The following enhancements are identified to evolve PulseCare 2026 from an academic prototype into a production-ready hospital management platform:

8.1 Database Persistence

The most critical improvement is replacing in-memory ArrayList storage with persistent database storage. Integrating Java Database Connectivity (JDBC) with MySQL or PostgreSQL would allow patient, doctor, and appointment records to persist across application sessions. An embedded H2 or SQLite database could be used for a portable, zero-configuration setup ideal for small clinics.

8.2 Multi-User Authentication and Role-Based Access

A login screen with secure password authentication and role-based access control (RBAC) would ensure that administrative operations are restricted to authorized personnel. Roles could include System Administrator (full access), Receptionist (patient and appointment management), Doctor (view-only access to their own appointments), and Billing Staff.

8.3 PDF Report and Invoice Generation

Adding a reporting module using iTextPDF or Apache PDFBox would enable staff to generate printed patient registration summaries, daily appointment schedules, and billing invoices. This is a critical real-world requirement for any hospital information system.

8.4 Electronic Health Records (EHR)

Extending the Patient model to support full EHR would include medical history, previous diagnoses, ongoing prescriptions, allergy records, uploaded lab reports (PDFs/images), and visit notes written by doctors. This transforms the system from a registry into a comprehensive clinical record tool.

8.5 Network and Cloud Capabilities

Migrating the application to a client-server architecture using Java RMI, REST APIs (Spring Boot), or WebSockets would enable multi-branch hospital management, remote access for doctors, and real-time synchronization of records across departments. Cloud deployment on AWS, Azure, or Google Cloud would add scalability and disaster recovery.

8.6 Advanced Analytics Dashboard

Replacing static dashboard widgets with interactive charts using JFreeChart or JavaFX would provide administrators with visual analytics including daily patient admission trends, doctor workload distribution, department-wise appointment volume, and peak hour analysis.

8.7 Mobile Application

A companion Android or iOS application allowing doctors to view their appointment schedules, patients to book and track their visits, and administrators to get real-time notifications would greatly extend the utility of the system.

8.8 Automated Notifications

Integrating JavaMail API or Twilio SMS service to send automated appointment reminders to patients, notifications to doctors about upcoming visits, and alerts for schedule changes would improve operational efficiency and patient satisfaction.

9. Conclusion

PulseCare 2026 is a comprehensive, fully functional Hospital Management System built entirely in Java using Swing for the graphical user interface. This project serves as both a practical demonstration of advanced Java programming skills and a meaningful contribution to the academic portfolio of its developers.

Through the development of this system, the team successfully applied and demonstrated core object-oriented programming principles including encapsulation through private fields and public getters in model classes, abstraction through clean method separation and component factory methods, and the practical use of Java's event-driven programming model through lambda-based action listeners and Swing event dispatch thread management.

The application features a modern, aesthetically refined user interface that stands apart from conventional academic projects. The use of custom gradient panels, color-coded status indicators, responsive layout management, and live search-and-filter capabilities elevates PulseCare 2026 to the standard of a professional prototype. The 20 pre-seeded doctors spanning diverse medical specializations and the two initial patients with appointments ensure an immediately compelling demo experience for evaluators.

From a software engineering perspective, the project maintains a clean separation of concerns: model classes (Patient, Doctor, Appointment) are entirely decoupled from the UI layer (HospitalManagementUI), and all data management logic is handled within clearly named private methods. This design facilitates straightforward extension and maintenance, forming a solid foundation for the future enhancements outlined in Section 8.

In conclusion, PulseCare 2026 fulfills all stated objectives of the Java Object-Oriented Designing and Programming course. It demonstrates mastery of OOP concepts, Swing GUI development, event-driven programming, and clean code architecture. The project is a testament to the team's ability to design, implement, and deliver a polished software product that addresses a real-world domain — hospital administration — using core Java technologies.

10. References

Official Documentation

- Oracle Corporation. (2023). Java SE 17 API Documentation. Retrieved from https://docs.oracle.com/en/java/javase/17/docs/api/
- Oracle Corporation. (2023). The Java Tutorials - Creating a GUI With Swing. Retrieved from https://docs.oracle.com/javase/tutorial/uiswing/
- Oracle Corporation. (2023). Java SE Documentation - java.awt Package. Retrieved from https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/java/awt/package-summary.html
- Oracle Corporation. (2023). javax.swing Package Documentation. Retrieved from https://docs.oracle.com/en/java/javase/17/docs/api/java.desktop/javax/swing/package-summary.html

Textbooks and Academic References

- Bloch, J. (2018). Effective Java (3rd Edition). Addison-Wesley Professional. ISBN: 978-0134685991.
- Horstmann, C. S. (2021). Core Java Volume I - Fundamentals (12th Edition). Pearson. ISBN: 978-0137673629.
- Deitel, P. & Deitel, H. (2020). Java How to Program, Early Objects (11th Edition). Pearson Education. ISBN: 978-0134743356.
- Liang, Y. D. (2019). Introduction to Java Programming and Data Structures, Comprehensive Version (12th Edition). Pearson. ISBN: 978-0136520238.

Online Tutorials and Resources

- Baeldung. (2023). Guide to Java Swing. Retrieved from https://www.baeldung.com/java-swing
- GeeksForGeeks. (2023). Java Swing Tutorial. Retrieved from https://www.geeksforgeeks.org/java-swing/
- Stack Overflow Community. (Various Years). Java Swing TableRowSorter with RowFilter. Retrieved from https://stackoverflow.com/
- JavaPoint. (2023). Java Object-Oriented Programming Tutorial. Retrieved from https://www.javatpoint.com/java-oops-concepts
- TutorialsPoint. (2023). Java Swing - JTable Component. Retrieved from https://www.tutorialspoint.com/swing/swing_jtable.htm

Development Tools and Environments

- JetBrains. (2023). IntelliJ IDEA Documentation. Retrieved from https://www.jetbrains.com/idea/documentation/

- Eclipse Foundation. (2023). Eclipse IDE Documentation. Retrieved from https://help.eclipse.org/
- Adoptium Working Group. (2023). Eclipse Temurin JDK 17. Retrieved from https://adoptium.net/

Project Files

- PulseCare 2026 Source Code: Java_Project.zip — Contains all .java source files under src/hospital/ package.
- Compiled Binaries: Located in the out/hospital/ directory as .class files.
- Build Script: run.bat — Windows batch script for compiling and running the application from command line.

GitHub Link

Team Member Github Link:

1. https://github.com/siddiquinadeem51/Hospital_Management_System.git
2. https://github.com/priyanshu159/Hospital-Management-System